

Applied Implicit Computational Complexity

Neea Rusch · 15 August 2025

Doctoral Dissertation Defense

I JUST WROTE THE
MOST BEAUTIFUL
CODE OF MY LIFE.



Example

Compare the two programs

Assume variables $X, Y, Z \in \mathbb{Z}$ and $\text{bit} \in \{0, 1\}$.

```
if (bit) { Z = X }  
else    { Z = Y }
```

vs.

$$Z = X * \text{bit} + Y * (1 - \text{bit})$$

Static program analysis

Many standard techniques:¹ data flow analysis, type systems, constraint-based analysis, abstract interpretation,...

My “toolbox” comes from **implicit computational complexity**.

¹ See, e.g., Anders Møller and Michael I Schwartzbach. *Static Program Analysis*. Department of Computer Science, Aarhus University, 2024, pp. 1–210. URL: <https://cs.au.dk/~amoeller/spa/spa.pdf> and Flemming Nielson, Hanne R Nielson, and Chris Hankin. *Principles of program analysis*. Springer, 2010, pp. 1–473. ISBN: 9783642084744.

Goals

1. What is implicit computational complexity?
2. How can we *apply* implicit computational complexity?
3. What did we learn from the research?

Applied

Implicit

Computational

Complexity

Applied

Implicit

Computational

solvable (in principle) by a computing device

Complexity

Applied

Implicit

Computational

solvable (in principle) by a computing device

Complexity

at what cost

Applied

Implicit

in terms of programming languages

Computational

solvable (in principle) by a computing device

Complexity

at what cost

Applied

how to use such theory

Implicit

in terms of programming languages

Computational

solvable (in principle) by a computing device

Complexity

at what cost

Implicit Computational Complexity (ICC)

Let L be a **programming language**, C a **complexity class**, and $\llbracket p \rrbracket$ the function computed by program p .

Find a **restriction** $R \subseteq L$, such that the following equality holds:

$$\{\llbracket p \rrbracket \mid p \in R\} = C$$

The variables L , C , and R are the parameters that vary greatly between different ICC systems.²

²Romain Péchoux. "Complexité implicite : bilan et perspectives". Habilitation à Diriger des Recherches (HDR). Université de Lorraine, Dec. 2020. URL: <https://hal.univ-lorraine.fr/tel-02978986v3/file/HDR-RP.pdf>.

Implicit Computational Complexity (ICC)

Programming language	C, Java, λ -calculus...
+ restriction	type system, syntax structure, data flow...
$\Rightarrow$ complexity class	PTIME, EXP, L, PP...

Restricting languages is a bit controversial



What are the practical limitations of a non-turing complete language?

Asked 14 years ago Modified 4 months ago Viewed 12k times



74



As there are non-Turing complete languages out there, and given I didn't study Comp Sci at university, could someone explain something that a Turing-incomplete language (like [Coq](#)) cannot do?

Practical non-Turing-complete languages?

Asked 15 years, 8 months ago Modified 10 years, 2 months ago Viewed 22k times



55



Nearly all programming languages used are [Turing Complete](#), and while this affords the language to represent any [computable](#) algorithm, it also comes with its own set of [problems](#). Seeing as all the algorithms I write are intended to halt, I would like to be able to represent them in a language that guarantees they will halt.

Programming languages with ~~restrictions~~ guarantees

(safe) Rust

free of memory errors and data races, with controlled aliasing



Total functional programming

all programs are provably terminating



Interactive theorem proving

require termination, but enable constructing formal proofs

Synchronous programming

for real-time reactive systems with response-time and memory restrictions

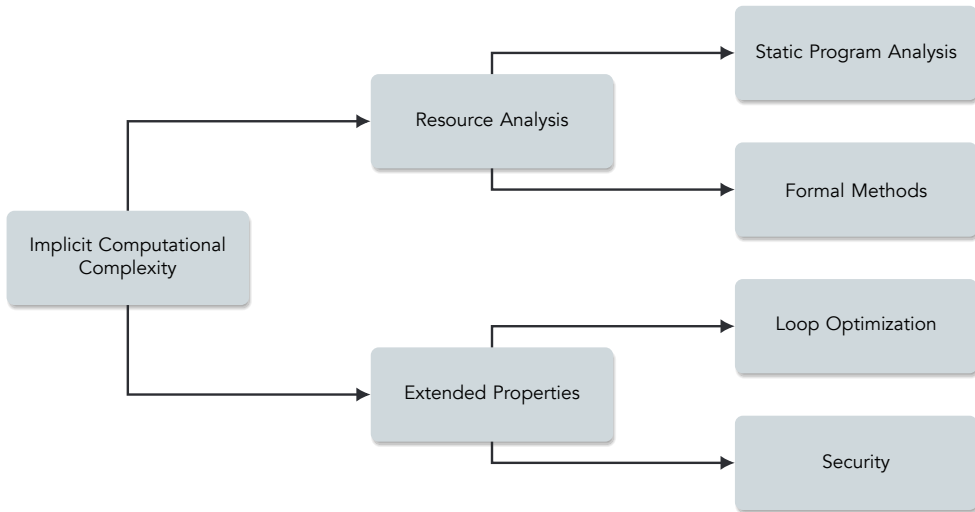
The (Applied) ICC Challenge

Despite compelling features, ICC remains largely a theoretical novelty. The practical power, limitations, and capabilities are not well-understood.

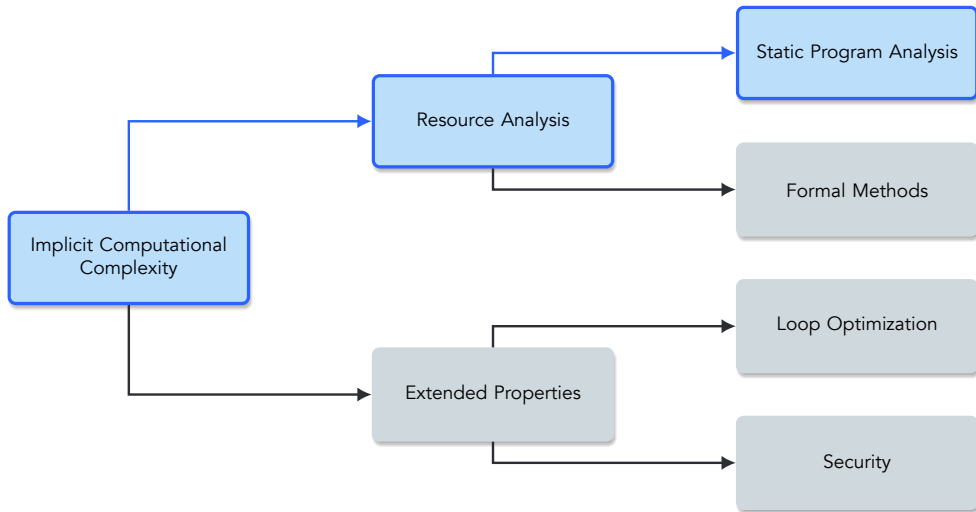
Hypothesis

Implicit computational complexity offers applied utilities when lifted from the theoretical domain.

The Big Picture



The Big Picture



Resource analysis

Apply ICC in designed way toward *automatic* resource analysis.

Some research questions:

- Can we develop practical resource analyses based on ICC theories?
- If theories can be automated, what are their use cases?
- Is the theory correct: can we formally prove, e.g., soundness?

The flow calculus of mwp-bounds³

Data flow analysis for certifying that **final values** computed by a deterministic imperative program will be **bounded by polynomials** in the program's **inputs**.

³Neil D. Jones and Lars Kristiansen. "A flow calculus of mwp-bounds for complexity analysis". In: *ACM Transactions on Computational Logic* 10.4 (Aug. 2009), 28:1–28:41. ISSN: 1557-945X. DOI: 10.1145/1555746.1555752.

The goal is to discover a polynomially bounded data-flow relation for command C , between its variables' initial values x_i and final values x'_i : $\llbracket C \rrbracket(x_i \rightsquigarrow x'_i)$.

$$C' \equiv \begin{array}{l} X_1 := X_2 + X_3; \\ X_1 := X_1 + X_1 \end{array}$$
$$\begin{array}{l} x'_1 \leq 2x_2 + 2x_3 \\ x'_2 \leq x_2 \\ x'_3 \leq x_3 \end{array}$$

✓ PASS

$$C'' \equiv \begin{array}{l} X_1 := 1; \\ \text{loop } X_2 \{X_1 := X_1 + X_1\} \end{array}$$
$$\begin{array}{l} x'_1 \leq 2^{x_2} \\ x'_2 \leq x_2 \end{array}$$

✗ FAIL

Mechanics of the flow calculus

1. Imperative programming language

(var) $X_1 \mid X_2 \mid X_3 \mid \dots$ (aexp) $e + e \mid e * e$ (bexp) $e = e \mid e < e \mid \dots$
(com) $\text{skip} \mid X := e \mid C;C \mid \text{if } b \text{ then } C \text{ else } C \mid \text{loop } X \{C\} \mid \text{while } b \text{ do } \{C\}$

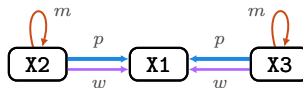
- A program is a sequence of commands.
- A program manipulates a fixed number of natural number variables.

Mechanics of the flow calculus

2. Flow coefficients (dependencies)

0 : no dependency m : maximal w : weak polynomial p : polynomial

weaker stronger

$$C' \equiv \begin{array}{l} X_1 := X_2 + X_3; \\ X_1 := X_1 + X_1 \end{array}$$


Mechanics of the flow calculus

3. Inference rules

$$\frac{}{\vdash x_i : \{\overset{m}{i}\}} \text{ E1} \quad \frac{\vdash x_i : V_1 \quad \vdash x_j : V_2}{\vdash x_i \star x_j : pV_1 \oplus V_2} \text{ E3} \quad \frac{\vdash e : V}{\vdash x_j = e : 1 \overset{j}{\leftarrow} V} \text{ A} \quad \dots$$

- 1 to 3 rules for each expression/command of the imperative language.
- The nondeterminism allows providing guarantees to more programs.

Mechanics of the flow calculus

4. mwp-bound

$$\max(\vec{x}, \text{poly}_1(\vec{y})) + \text{poly}_2(\vec{z})$$

- The symbols $\vec{x}, \vec{y}, \vec{z}$ are disjoint variable lists.
Variables characterized by m -flow are in $\vec{x}$, w -flow in $\vec{y}$, and p -flow in $\vec{z}$.
- The poly_1 and poly_2 are honest polynomials.
They are built up of variables, constants, and operators $+$ and $\times$.

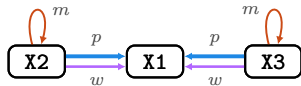
Mechanics of the flow calculus

In summary:

1. Program in imperative programming language (input)
2. Coefficients (track dependence)
3. Inference rules (logic)
4. mwp-bounds (output)

Derivation success

$C' \equiv$
 $X_1 := X_2 + X_3;$
 $X_1 := X_1 + X_1$



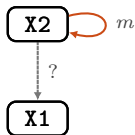
$$\begin{array}{c}
 \frac{}{\vdash X_2 : \begin{pmatrix} 0 \\ m \\ 0 \end{pmatrix}} E1 \quad \frac{}{\vdash X_3 : \begin{pmatrix} 0 \\ 0 \\ m \end{pmatrix}} E1 \\
 \hline
 \vdash X_2 + X_3 : \begin{pmatrix} 0 \\ p \\ m \end{pmatrix} E3 \\
 \hline
 \vdash X_1 := X_2 + X_3 : \begin{pmatrix} 0 & 0 & 0 \\ p & m & 0 \\ m & 0 & m \end{pmatrix} A \\
 \vdots \\
 \hline
 \vdash X_1 := X_1 + X_1 : \begin{pmatrix} p & 0 & 0 \\ 0 & m & 0 \\ 0 & 0 & m \end{pmatrix} A \\
 \vdots \\
 \hline
 \vdash X_1 := X_2 + X_3; X_1 := X_1 + X_1 : \begin{pmatrix} 0 & 0 & 0 \\ p & m & 0 \\ p & 0 & m \end{pmatrix} C
 \end{array}$$

$$x_1' \leq x_2 + x_3 \quad \wedge \quad x_2' \leq x_2 \quad \wedge \quad x_3' \leq x_3$$

Derivation failure

$$\forall i, M_{ii}^* = m \frac{\vdash c : M}{\vdash \text{loop } x_\ell \{c\} : M^* \oplus \{ \overset{p}{\ell} \rightarrow j \mid \exists i, M_{ij}^* = p \}} \text{L}$$

$C'' \equiv$ $x_1 := 1;$
 $\text{loop } x_2 \{x_1 := x_1 + x_1\}$



$$\frac{}{\vdash x_1 := 1 : \begin{pmatrix} m \\ 0 \end{pmatrix}} \text{E1}$$

$$\vdots$$

$$\frac{}{\vdash x_1 := x_1 + x_1 : \begin{pmatrix} p & 0 \\ 0 & m \end{pmatrix}} \text{A}$$

$$\vdots$$

$$\perp$$

Derivation ?

```
if (bit) { Z = X }  
else     { Z = Y }
```

VS.

$$Z = X * \text{bit} + Y * (1 - \text{bit})$$

Flow calculus application challenges

- How to handle **nondeterminism**?
- How to handle **derivation failure**?

The non-determinism problem

Analysis of expression $X_2 + X_3$ has 3 derivations.

$$\frac{}{\vdash e : \{i^w \mid X_i \in \text{var}(e)\}} \text{E2} \quad \text{by inference rule E2} \quad \begin{pmatrix} 0 \\ w \\ w \end{pmatrix}$$

$$\frac{\vdash X_i : V_1 \quad \vdash X_j : V_2}{\vdash X_i \star X_j : pV_1 \oplus V_2} \text{E3} \quad \text{by inference rules E1 and E3} \quad \begin{pmatrix} 0 \\ p \\ m \end{pmatrix}$$

$$\frac{\vdash X_i : V_1 \quad \vdash X_j : V_2}{\vdash X_i \star X_j : V_1 \oplus pV_2} \text{E4} \quad \text{by inference rules E1 and E4} \quad \begin{pmatrix} 0 \\ m \\ p \end{pmatrix}$$

In general, n choices yields 3^n derivations.

FIX: Handling non-determinism

Idea: track derivation choices as functions from choices to coefficients.

- ▷ If a coefficient depends on a choice: represent as 3 elements.
- ▷ If independent: represent as a single element.

We track not only dependencies, but a history of derivation choices.

Internalized non-determinism

$$X_1 = X_2 + X_3$$

↓

$$\left(\begin{array}{ccc} m & 0 & 0 \\ m & m & 0 \\ p & 0 & m \end{array} \right) \quad \left(\begin{array}{ccc} m & 0 & 0 \\ p & m & 0 \\ m & 0 & m \end{array} \right) \quad \left(\begin{array}{ccc} m & 0 & 0 \\ w & m & 0 \\ w & 0 & m \end{array} \right)$$

↓

$$\left(\begin{array}{ccc} m & 0 & 0 \\ m.\delta(0,0)+p.\delta(1,0)+w.\delta(2,0) & m & 0 \\ p.\delta(0,0)+m.\delta(1,0)+w.\delta(2,0) & 0 & m \end{array} \right)$$

The failure problem

$C \equiv \text{while } (b) \text{ do } \{X1 := X2 + X2\}$

Derivation of $X1 := X2 + X2$ has two matrices: $\begin{pmatrix} 0 & 0 \\ p & m \end{pmatrix}$ and $\begin{pmatrix} 0 & 0 \\ w & m \end{pmatrix}$.

The inference rule for analyzing an unbounded loop is

$$\forall i, M_{ii}^* = m \text{ and } \forall i, j, M_{ij}^* \neq p \quad \frac{\vdash C : M}{\vdash \text{while } b \text{ do } \{C\} : M^*} \quad W$$

Choosing $\begin{pmatrix} 0 & 0 \\ p & m \end{pmatrix}$ fails, but choosing $\begin{pmatrix} 0 & 0 \\ w & m \end{pmatrix}$ succeeds.

FIX: New way to represent failure

Idea: We introduce ∞ flow to represent non-polynomial dependencies.

0 : none m : maximal w : weak polynomial p : polynomial ∞ : **failure**

Every derivation can be completed without restarts.

Captures localized information about where failure occurs.

Once failure is introduced, it cannot be erased: $\infty \times^\infty 0 = \infty$.

$$C \equiv \text{while } (b) \text{ do } \{X_1 := X_2 + X_2\} \quad \begin{pmatrix} m + \infty\delta(0,0) + \infty\delta(1,0) & 0 \\ \infty\delta(0,0) + \infty\delta(1,0) + w\delta(2,0) & m \end{pmatrix}$$

What did we learn?

The enhanced flow calculus

Q: Can we develop practical resource analyses based on ICC theories?

A: Yes, eventually.

- All derivations captured deterministically in one (complex) matrix.
- The mwp-bounds are “hidden” (temporarily).
- The enhanced system gives more information than the original system.



```
$ pip install pymwp
```

```
Successfully installed pymwp!
```

```
$ pymwp notinfinite_3.c
```

```
INFO (result): Bound:
```

```
 $X0' \leq \max(X0, X1) + X2 * X3$ 
```

```
 $\wedge X1' \leq X1 + X2$ 
```

```
 $\wedge X2' \leq X2 + X3$ 
```

```
 $\wedge X3' \leq X3$ 
```

ICC applications

Q: If theories can be automated, what are their use cases?

A: There are *several* applications.

- Automatic static resource analysis, e.g., pymwp⁴.
- Specification inference: use obtained bounds in verification.
- The likely pathway toward formal reasoning about complexity.
- ...and many more.

⁴Clément Aubert et al. "pymwp: A Static Analyzer Determining Polynomial Growth Bounds". In: *Automated Technology for Verification and Analysis*. Springer Nature Switzerland, 2023, pp. 263–275. ISBN: 9783031453328. DOI: 10.1007/978-3-031-45332-8_14.

Certifying ICC theories

Q: Is the theory correct, and can we formally prove, e.g., soundness?

A: With high confidence, *likely* yes.

- Added confidence in the correctness of the informal (paper) proofs.
- Evidence of utility to justify the formalization effort.
- Requires making explicit the details of the informal proofs.

The soundness theorem of mwp-calculus. Letting C be a program and M and an mwp-matrix, $\vdash C : M$ implies $\models C : M$.

Certifying ICC theories

LEMMA 7.2 SPLITTING. For any honest polynomial $p(\vec{x}, \vec{y})$ there exist honest polynomials $p_1(\vec{x})$ and $p_2(\vec{y})$ such that $p(\vec{x}, \vec{y}) \leq p_1(\vec{x}) + p_2(\vec{y})$.

PROOF. Recall that an honest polynomial is build up from constants in $\mathbb{N}$ and variables by applying the operators $+$ and $\times$. We prove the lemma by induction on the structure of an honest polynomial.

The lemma is obvious when p is a constant or a single variable. Now, assume that $p(\vec{x}, \vec{y}) = q(\vec{x}, \vec{y}) \times r(\vec{x}, \vec{y})$. By the induction hypothesis we have honest polynomials q_1, q_2, r_1, r_2 such that $q(\vec{x}, \vec{y}) \leq q_1(\vec{x}) + q_2(\vec{y})$ and $r(\vec{x}, \vec{y}) \leq r_1(\vec{x}) + r_2(\vec{y})$. Observe that for any u, v we have

$$u \times v \leq \max(u, v)^2 \leq \max(u^2, v^2) \leq u^2 + v^2. \quad (*)$$

Thus, we have

$$\begin{aligned} p(\vec{x}, \vec{y}) &= q(\vec{x}, \vec{y}) \times r(\vec{x}, \vec{y}) \leq (q_1 + q_2) \times (r_1 + r_2) \\ &= q_1 r_1 + q_1 r_2 + q_2 r_1 + q_2 r_2 \\ &\leq q_1^2 + r_1^2 + q_1^2 + r_2^2 + q_2^2 + r_1^2 + q_2^2 + r_2^2 \quad (*) \\ &= 2q_1^2 + 2r_1^2 + 2q_2^2 + 2r_2^2 \end{aligned}$$

and the lemma holds when $p_1 = 2q_1^2 + 2r_1^2$ and $p_2 = 2q_2^2 + 2r_2^2$.

The case when $p(\vec{x}, \vec{y})$ is of the form $q(\vec{x}, \vec{y}) + r(\vec{x}, \vec{y})$ is easy, and we omit the details. $\square$

Lemma splitting p:

$$\exists p_1 p_2, \text{eval st } p \leq \text{eval st } p_1 + \text{eval st } p_2.$$

Proof.

```
elim: p => [n|x|????]? [q1[q2 Ht1]]? [r1[r2 Ht2]]] /=.
- ∃ n, 0; by rewrite addn0 leqnn.
- ∃ x, 0; by rewrite addn0 leqnn.
- eexists; eexists; by [].
- ∃ ((eval st q1^2).*2 + (eval st r1^2).*2),
  ((eval st q2^2).*2 + (eval st r2^2).*2) => /=.
  rewrite (leq_trans (leq_mul Ht1 Ht2)) // {Ht1 Ht2}.
  rewrite !mulnDl !mulnDr -addnA.
  apply: (leq_trans(leq_mulsqn _ _ _)).
  rewrite addnCA.
  apply: (leq_trans(leq_mulsqn _ _ _)).
  rewrite addnAC -!addnn -!addnA
    !leq_add2l !addnA leq_add2r.
  apply: (leq_trans(leq_mulsqn _ _ _)).
  apply: (leq_trans(leq_mulnsq _ _ _)).
  by rewrite !leq_add2r addnC.
```

Qed.

informal



formal

Analyzing extended properties

Q: Can we use ICC to track *other* properties, beyond resources?

A: Yes, with adjustment.

- Some ICC systems are flexible – can track other properties.
- Complexity guarantee may be lost, but something else is gained.
- Develops deeper understanding of the theory.
- Pushes to introduce ICC to new research communities.

Takeaway messages

Implicit computational complexity

1. offers complementary techniques for resource analysis
2. analyses can be adjusted to track other program properties
3. has potential to guarantee properties by construction

Implicit computational complexity is much broader in utility than the name implies